1. Introduction

Competition 2 focuses on the challenge of detecting AI-generated text, which is becoming increasingly critical as a device for fraud, in domains like impersonation, spreading of misinformation, and social engineering attacks. The competition frames this task as a binary classification task, with the objective of accurately identifying whether a human or an AI model authored a given paragraph. My approach to this competition progresses from exploratory data analysis (EDA), data processing and feature engineering, developing and validating models, additional techniques to help improve the model, and model evaluations to determine which final model to choose. For more details on the process, refer to the notebook E1121498.ipynb.

2. Methodology

2.1. Exploratory Data Analysis

The dataset consisted of 4,712 text samples, labelled as human-written (0) or AI-generated (1), with a class imbalance of approximately 69% human to 31% AI. There were also approximately 500 duplicate samples, all of which belonged to the human class. The duplicated data would have to be handled when creating splits or cross-validation folds later on. I decided to just drop the 500 samples, as it would help us slightly with the class imbalance down to 65% human and 35% AI. Unfortunately, I did not discover the duplicated data in my initial EDA until much later, therefore most of the experiments were done on the base dataset, while some of them were re-run on deduplicated data. There will be more discussion on this below. Apart from that, not much more EDA can be done without any feature engineering, as there is only the text column and the label.

2.2. Data Preprocessing and Feature Engineering

In this section, I engineered some features and performed further data analysis on these engineered features. From the text column, linguistic features were extracted, through basic text features, readability features, and parts of speech features, a total of 33 engineered features. Basic features included counts of words, unique words, sentences, and averages of these among others. Out of the basic features, average sentence length, word count and type token ratio (unique words divided by total words or lexical diversity) turned out to be extremely strong features for many models. The readability features included Flesch Reading Ease score, grade level readability, and Shannon entropy. Flesch reading ease turned out to have the highest feature importance for most models. Lastly, spaCy's en_core_web_sm model was used for POS (parts of speech) tagging, of which I retained all 14 classes within my feature space, which I found to have helped my models significantly. I wrote a concise feature engineering pipeline for reproducibility and saved these features to a new CSV file to avoid rerunning this pipeline every time I train a model. As much as possible, data leakage was avoided by computing features only from the text sample and row-wise, using pre-defined resources (nltk corpora) and formula-based calculations that did not depend on other rows.

Most importantly, I found significant statistical predictors of label within these engineered features, learning that AI texts showed lower minimum word counts and character lengths, a greater lexical diversity, more complex text with lower reading ease scores, and a higher shannon entropy (less

predictable maybe due to the data imbalance). I performed t-tests with the features against label, and most of them were found to have statistically significant means between human and AI text.

2.3. Model Development and Validation

For model development, all experiments used Scikit-Learn's stratified k-fold cross validation, with the same random state to ensure that all splits are the same. This allows me to fairly compare models later on, and ensures reproducibility and fold alignment for ensembling later on. Having stratified splits allows me to preserve the class distribution across folds so that models do not overfit to a certain fold's label distribution. While the cross validation was done by ensembling the five models generated at each fold, allowing me to use the full 100% of the training set in my final output models, while avoiding overfit. Regarding the duplicated data mentioned earlier, I would have considered stratified group k-fold cross validation, but I already started experimenting, and did not want to bring in added complexity in my model evaluations later on. Having the models all train on the same splits should give all models a similar advantage with any data leakage caused by the duplicates, allowing me to still fairly evaluate them. On top of that, I used pipelines wherever I can, to avoid data leakage and fitting on test data, and used Optuna for all hyperparameter optimizations with MedianPruner for early stopping and the objective of maximising ROC-AUC score.

I started off with logistic regression and ridge, both relatively similar but with different optimization techniques. These models were trained on TF-IDF features only, which was done carefully at every fold, since it uses cross-sample statistics. They both performed well, and did even better with minor hyperparameter tuning. It was interesting to find that an ngram range of (1,1) worked best for the TF-IDF on log regression, showing that the words themselves maybe matter more than the context around it. Though a range of (1,5) was best for the ridge regression.

Next, I tested out tree-based models, mainly XGBoost. Here I was focusing on testing the usefulness of my engineered features, since I was afraid to use them on my linear models given the high correlations between some of them, and since a lot of them were far from a normal distribution. I ran a version of just the TF-IDF features just like the linear models, but also did another one on the TF-IDF features alongside my engineered features but with TruncatedSVD to reduce dimensionality. I used the feature importances to experiment with my feature set initially, but ended up not dropping any features since any individual change in my engineered feature set had marginal performance change. From these experiments, I found XGBoost with TF-IDF and engineered features alongside Optuna hyperparameter tuning performed the best so far.

Due to the improvements when using all features, I wanted to try other model architectures on the same feature set, starting with neural networks. The first model was MLP, using the same features as the XGBoost model, TruncatedSVD on TF-IDF features, and engineered features, only this time with StandardScalar to normalize the structured features. Next were other architectures like CharCNN and BiLSTM, run only on the text itself, both capturing different aspects, like CharCNN focusing more on local features while BiLSTM captures longer range contextual dependencies and positional information. Wherever applicable, tools like early stopping were used to prevent overfitting, model checkpointing to recall the best weights in the case of divergence during training, and learning rate scheduling (ReduceLROnPlateau) to improve convergence and generalization. All of these models

also had dropout layers for regularization and batch normalization for MLP. Of these models, Optuna-tuned MLP performed best, followed by CharCNN then BiLSTM. The order of the latter two aligned with our experimentation with linear models earlier, finding that local patterns (lower ngram ranges) worked better.

Next, hybrid NN architectures were used, combining the strongest MLP with each of the text based models. This was done via parallel branches that fed into a concatenation layer and then classified from there. The same techniques were used as well, and considerable improvement was made when including CharCNN, and a slightly smaller improvement between BiLSTM and the hybrid with BiLSTM and MLP. Up till here, hybrid CharCNN with MLP is the best performing model.

I then experimented with transfer learning, using BERT based transformers like RoBERTa and DeBERTa. Both performed extremely well, significantly outperforming my best models. I found DeBERTa to perform better than RoBERTa, possibly due to its disentangled attention mechanism, which separates positional attention with content-based attention. It is possible that the content based attention is more important as per our findings with the TF-IDF, and preserving that in a separate attention layer might help. With these transformer models, I used early stopping, learning rate scheduling and regularization, alongside class weighting and gradient clipping to avoid overfitting and ensure the training process goes smoothly.

Lastly I experimented with ensembles, using my best models while ensuring diversity checking on model and submission file correlations and also based on performance. It came down to ridge, XGBoost, hybrid CharCNN and MLP, and DeBERTa. I started off with a simple average ensemble and a meta logistic regression ensemble, but the simple average performed better even despite Optuna tuning the meta learner. Since I generated the out-of-fold predictions for each model with the initial ensembles, I saved them to a csv file, and was able to quickly generate different ensembles quickly. With this ability, I experimented with weighted average ensembles, trying weights based on individual CV ROC-AUC scores, or maximising ROC-AUC with the weights, or minimizing LogLoss. The min log loss version performed best in the validation, and in the Kaggle submission.

3. Results and Discussion

3.1. Model Evaluations and Final Strategy

Ultimately, the ensembles outperformed all individual models, apart from DeBERTa. Therefore, I chose to use the min Log Loss ensemble, alongside DeBERTa. I made sure to retrain these models on the deduplicated data and resubmitted them. The majority of the model evaluations were done during and right after training of individual models, in order to gather as much understanding of my models as I could, to allow for effective ensembling.

That said, I still prepared a reusable evaluation notebook that would detect models and metrics in my directory structure. I found that the original models all performed slightly better than the models on deduplicated data in Kaggle submission and CV, and that Optuna optimization was quite minor with a 0.1-0.2% gain. Apart from that, all models performed well (above 97% ROC AUC), and therefore the selection of models had to take into consideration other factors like generalization and

model diversity on top of ROC-AUC score which will be used for grading. Generalization was calculated by comparing Kaggle score to CV score, and trying to see whether the model really performs the same over test and train or if there was overfit to a certain set.

4. Conclusion and Reflection

To conclude, this competition was rather difficult at the beginning with the lack of any structured features, but provided me opportunities to discover textual engineering methods, with NLTK, Textstat, spaCy, etc. to create unique and interesting features (refer to table 1) I never knew about before. I was also able to structure my experiments much better than in competition 1, and tracking of all metrics and even saving models for each fold helped me significantly during model ensembling. Though I regret not checking for duplicated data earlier on, I stuck to trusting my CV score comparisons given my rigorous tracking (refer to table 2), and only retraining what I needed to. Apart from that, I would also have done group stratified k-fold cross validation from the start if I found out about the duplicates, since models were performing better before deduplication even on the Kaggle leaderboard. Another note was that I tried to avoid trusting only the Kaggle scores when evaluating my models since it consisted of only 30% of the ~1300 test data, which is quite small compared to 1000 CV validation samples.

ID	Feature	Category	Source
1	char_len	Basic Text Features	Base Python
2	word_count	Basic Text Features	Base Python
3	unique_word_count	Basic Text Features	Base Python
4	type_token_ratio	Basic Text Features	Base Python
5	sentence_count	Basic Text Features	Base Python
6	avg_sentence_length	Basic Text Features	Base Python
7	punctuation_count	Basic Text Features	Base Python
8	punctuation_ratio	Basic Text Features	Base Python
9	uppercase_count	Basic Text Features	Base Python
10	uppercase_ratio	Basic Text Features	Base Python
11	stopword_count	Basic Text Features	NLTK corpus + Base Python
12	stopword_ratio	Basic Text Features	NLTK corpus + Base Python
13	flesch_reading_ease	Readability Features	Textstat
14	flesch_kincaid_grade	Readability Features	Textstat
15	shannon_entropy	Readability Features	Base Python
16	pos_adj	POS Features	SpaCy en_core_web_sm POS tagging model

Table 1: Feature List (POS features truncated)

	Submission Score	CV Score on Train					
Experiment Name	ROC-AUC	Precision	Recall	F1-Score	ROC-AUC	Accuracy	Features
exp_1_tfidf_log	0.98954	0.983805	0.820945	0.895027	0.992981	0.93949	TF-IDF on text
exp_1_tfidf_log_optuna	0.99611	NA	NA	NA	NA	NA	TF-IDF on text
exp_2_tfidf_ridge	NA	0.992908	0.945946	0.968858	0.998752	0.980912	TF-IDF on text
exp_2_tfidf_ridge_optuna	0.99583	NA	NA	NA	NA	NA	TF-IDF on text
exp_3_struct_xgboost	NA	0.913748	0.847876	0.879398	0.973544	0.927209	Structured features
exp_3_struct_xgboost_optuna	0.97949	0.928826	0.881757	0.904679	0.986471	0.941614	Structured features
exp_4_all_xgboost	NA	0.957269	0.907359	0.93157	0.991444	0.958193	TF-IDF -> TruncatedSVD + Structured
exp_4_all_xgboost_optuna	0.99736	0.979381	0.962838	0.971039	0.99851	0.981953	TF-IDF -> TruncatedSVD + Structured
exp_5_all_mlp	0.99685	0.952668	0.93712	0.944807	0.993332	0.96562	TF-IDF -> TruncatedSVD + Structured -> imputer + scalar
exp_5_all_mlp_optuna	0.9985	0.965615	0.947256	0.956332	0.99624	0.972835	TF-IDF -> TruncatedSVD + Structured -> imputer + scalar
exp_6_BiLSTM	0.99178	0.954002	0.949963	0.951921	0.993287	0.969861	Text
exp_7_CharCNN	0.9952	0.935104	0.891154	0.912162	0.984769	0.946095	Text
exp_8_hybrid_charcnn_mlp	0.99886	0.964657	0.940497	0.952392	0.994831	0.970501	all_mlp + charcnn features
exp_9_hybrid_bilstm_mlp	0.9932	0.970169	0.94589	0.957825	0.995552	0.973894	all_mlp + bilstm features
exp_10_log_stacked_ensemble	0.99905	0.971067	0.95199	0.961408	0.997934	0.97602	exp_8, exp_4_optuna, and exp_2_optuna predictions
exp_10_weighted_avg_ensemble	0.99893	0.997657	0.960274	0.947938	0.972935	0.975382	exp_8, exp_4_optuna, and exp_2_optuna predictions
exp_11_roberta	0.99874	0.961878	0.993912	0.977517	0.999346	0.98557	Text
exp_12_deberta	0.99964	0.970978	0.993238	0.981964	0.999397	0.98854	Text

Table 2: Model Tracking and Performance